Exp 1

```
#include <stdio.h>
int main() {
 int arr[100], size, i, j, temp;
 printf("Enter number of elements: ");
 scanf("%d", &size);
 printf("Enter %d elements:\n", size);
 for(i = 0; i < size; i++) {
 scanf("%d", &arr[i]);
 }
 for(i = 1; i < size; i++) {
 temp = arr[i];
 j = i - 1;
 while(j >= 0 && arr[j] > temp) {
 arr[j + 1] = arr[j];
 j--;
 }
 arr[j + 1] = temp;
 }
 printf("\nSorted Array: ");
 for(i = 0; i < size; i++) {
 printf("%d ", arr[i]);
 }
 return 0;
}
```

Output

```
Enter number of elements: 6
Enter 6 elements:
12
5
18
2
9
7
Sorted Array: 2 5 7 9 12 18
```

Algo:

1. Start.
2. Initialize array A and find n.
3. For j = 1 to n - 1:
- **key = A[j]**
- **i = j - 1**
- **While i ≥ 0 and A[i] > key:**
  - **Shift A[i] to A[i + 1]**

- Decrement i
- Insert key at A[i + 1].

4. Repeat for all elements.
5. Print the sorted array.
6.Stop

Exp 2

```
#include <stdio.h>
int binarySearch(int arr[], int left, int right, int key)
{
 while(left <= right)
 {
 int mid = left + (right - left) / 2;
 if(arr[mid] == key)
 return mid;
 if(arr[mid] < key)
 left = mid + 1;
 else
 right = mid - 1;
 }
 return -1;
}
int main()
{
 int arr[] = {10, 20, 30, 40, 50, 60, 70};
 int n = sizeof(arr) / sizeof(arr[0]);
 int key;
 printf("Enter element to search: ");
 scanf("%d", &key);
 int pos = binarySearch(arr, 0, n - 1, key); if(pos == -1)
 printf("Element not found.");
 else
 printf("Element found at index %d.", pos);
 return 0;
}
```

Output

```
PS C:\Users\rajveer\Documents\AOA> gcc binary.c -o binary
PS C:\Users\rajveer\Documents\AOA> .\binary
Enter element to search: 50
Element found at index 4.
PS C:\Users\rajveer\Documents\AOA>
```

Algo:

1 Algorithm BinSearch(a, n, x)

```
2  // Given an array a[1 : n] of elements in nondecreasing
3  // order, n >= 0, determine whether x is present, and
4  // if so, return j such that x = a[j]; else return 0.
5  {
6      low := 1; high := n;
7      while (low <= high) do
8      {
9          mid := ⌊(low + high)/2⌋;
10         if (x < a[mid]) then high := mid - 1;
11         else if (x > a[mid]) then low := mid + 1;
12         else return mid;
13     }
14     return 0;
15 }
```

Exp 3

```
#include <stdio.h>
#include <stdlib.h>
int i, j, k, a, b, u, v, n, ne = 1;
int min, mincost = 0, cost[9][9], parent[9];
int find(int);
int uni(int, int);
void main()
{
 printf("\n\n\tImplementation of Kruskal's algorithm\n\n");
 printf("Enter the no. of vertices\n");
 scanf("%d", &n);
 printf("\nEnter the cost adjacency matrix\n");
 for (i = 1; i <= n; i++)
 {
 for (j = 1; j <= n; j++)
 {
 scanf("%d", &cost[i][j]);
 if (cost[i][j] == 0)
 cost[i][j] = 999;
 }
 }
 printf("\nThe edges of Minimum Cost Spanning Tree are\n\n");
 while (ne < n)
 {
 for (i = 1, min = 999; i <= n; i++)
 {
 for (j = 1; j <= n; j++)
 {
 if (cost[i][j] < min)
 {
 min = cost[i][j]; a = u = i;
```

```
 b = v = j;
 }
 }
 }
 u = find(u);
 v = find(v);
 if (uni(u, v))
 {
 printf("\n%d edge (%d,%d) =%d\n", ne++, a, b, min);
 mincost += min;
 }
 cost[a][b] = cost[b][a] = 999;
 }
 printf("\n\tMinimum cost = %d\n", mincost);
}
int find(int i)
{
 printf("finding-i %d \n", i);
 while (parent[i])
 {
 i = parent[i];
 }
 printf("HIIIIIIIIIIIIIII %d \n", i);
 return i;
}
int uni(int i, int j)
{
 printf("m in uni %d %d \n", i, j);
 if (i != j)
 {
 printf("m in uni_1%d %d \n", i, j);
 parent[j] = i;
 printf("m in uni_parent %d \n", parent[j]);
 return 1;
 }
 return 0;
}
```

Output

```
PS C:\Users\admin\Desktop\aoa> gcc kruskal.c -o kruskal
PS C:\Users\aoa\Desktop\aoa> .\kruskal.exe
 Implementation of Kruskal's algorithm
Enter the no. of vertices
4
Enter the cost adjacency matrix
0 10 6 5
10 0 15 4
```

6 15 0 3
5 4 3 0
The edges of Minimum Cost Spanning Tree are
finding-i 3
HIIIIIIIIIIIIIII 3
finding-i 4
HIIIIIIIIIIIIIII 4
m in uni 3 4
m in uni_13 4
m in uni_parent 3
1 edge (3,4) =3
finding-i 2
HIIIIIIIIIIIIIII 2
finding-i 4
HIIIIIIIIIIIIIII 3
m in uni 2 3
m in uni_12 3
m in uni_parent 2
2 edge (2,4) =4
finding-i 1
HIIIIIIIIIIIIIII 1
finding-i 4 HIIIIIIIIIIIIIII 2
m in uni 1 2
m in uni_11 2
m in uni_parent 1
3 edge (1,4) =5
Minimum cost = 12
PS C:\Users\admin\Desktop\aoa>

Algo:

Kruskal's algorithm also follows greedy approach, which finds an optimum solution at every stage instead of focusing on a global optimum.
The sequence of steps for Kruskal's algorithm is given as follows:
1. First sort all the edges from the lowest weight to highest.
2. Take edge with the lowest weight and add it to the spanning tree. If the cycle is created, discard the edge.
3. Keep adding edges like in step 1 until all the vertices are considered.

Analysis of algorithms

pascal

```
Procedure kruskal(G)
   G - input graph :
begin
   A = ∅
   For each vertex v ∈ G.V:
```

```
      MAKE-SET(v)
   For each edge (u, v) ∈ G.E ordered by weight in increasing order (u, v):
      if FIND-SET(u) ≠ FIND-SET(v):
         A = A ∪ {(u, v)}
         UNION(u, v)
   return A
end kruskal;
```

Exp 4

```
#include <stdio.h>
struct item
{
 int profit;
 int weight;
 float ratio;
};
int main()
{
 int n, m, i, j;
 float totalProfit = 0;
 struct item temp;
 printf("\n\tKnapsack Problem using Greedy Method\n\n");
 printf("Enter the number of items: ");
 scanf("%d", &n);
 printf("Enter the capacity of knapsack: ");
 scanf("%d", &m);
 struct item a[n];
 printf("\nEnter the profits of items:\n");
 for (i = 0; i < n; i++)
 {
 printf("Profit of item %d: ", i + 1);
 scanf("%d", &a[i].profit);
 }
 printf("\nEnter the weights of items:\n");
 for (i = 0; i < n; i++)
 {
 printf("Weight of item %d: ", i + 1);
 scanf("%d", &a[i].weight);
 }
 /* Calculate profit/weight ratio */
 for (i = 0; i < n; i++)
 {
 a[i].ratio = (float)a[i].profit / a[i].weight;
 }
 /* Sort according to profit/weight ratio */
 for (i = 0; i < n - 1; i++)
 {
```

```
 for (j = i + 1; j < n; j++)
 { if (a[i].ratio < a[j].ratio)
 {
 temp = a[i];
 a[i] = a[j];
 a[j] = temp;
 }
 }
 }
 printf("\nItems sorted according to profit/weight ratio:\n");
 printf("Profit\tWeight\tRatio\n");
 for (i = 0; i < n; i++)
 {
 printf("%d\t%d\t%.2f\n",
 a[i].profit, a[i].weight, a[i].ratio);
 }
 printf("\nSelected items:\n");
 for (i = 0; i < n; i++)
 {
 if (a[i].weight <= m)
 {
 printf("Item with profit %d and weight %d -> 100%% selected\n",
 a[i].profit, a[i].weight);
 m = m - a[i].weight;
 totalProfit = totalProfit + a[i].profit;
 }
 else
 {
 float fraction = (float)m / a[i].weight;
 printf("Item with profit %d and weight %d -> %.2f%% selected\n",
 a[i].profit, a[i].weight, fraction * 100);
 totalProfit = totalProfit + (a[i].profit * fraction);
 break;
 }
 }
 printf("\nMaximum Profit = %.2f\n", totalProfit);
 return 0;
}
```

Output

```
PS C:\Users\admin\Desktop\AOA> gcc knapsack_greedy.c -o knapsack_greedy
PS C:\Users\admin\Desktop\AOA> .\knapsack_greedy.exe
 Knapsack Problem using Greedy Method
Enter the number of items: 7
Enter the capacity of knapsack: 15
Enter the profits of items:
Profit of item 1: 10
```

Profit of item 2: 5
Profit of item 3: 15
Profit of item 4: 7
Profit of item 5: 6
Profit of item 6: 18
Profit of item 7: 3
Enter the weights of items:
Weight of item 1: 2
Weight of item 2: 3
Weight of item 3: 5
Weight of item 4: 7
Weight of item 5: 6
Weight of item 6: 4
Weight of item 7: 1
Maximum Profit = 51.00
PS C:\Users\admin\Desktop\AOA>

Algo:

{
For j \(\leftarrow \) 1 to n
do X[j] \(\leftarrow \) 0
profit \(\leftarrow \) 0 // Total profit of item filled in the
knapsack weight \(\leftarrow \) 0 // Total weight of items
packed in knapsacks j \(\leftarrow \) 1
While (Weight <= M)
{
if (weight + W[j] =< M)
Analysis of algorithms
X[j] = 1
weight = weight +
W[j] else{
X[j] = (M -
weight)/w[j] weight
= M
}
Profit = profit + p[j] * X[j] j++;
}
}

Exp 5

```
#include <stdio.h>
struct Pair
{
 int min;
 int max;
};
```

```
struct Pair minMax(int a[], int low, int high)
{
 struct Pair result, left, right;
 int mid;
 // Only one element
 if (low == high)
 {
 result.min = a[low];
 result.max = a[low];
 return result;
 }
 // Two elements
 if (high == low + 1)
 {
 if (a[low] < a[high])
 {
 result.min = a[low];
 result.max = a[high];
 }
 else
 {
 result.min = a[high];
 result.max = a[low];
 }
 return result;
 }
 // Divide
 mid = (low + high) / 2; // Conquer
 left = minMax(a, low, mid);
 right = minMax(a, mid + 1, high);
 // Combine for minimum
 if (left.min < right.min)
 result.min = left.min;
 else
 result.min = right.min;
 // Combine for maximum
 if (left.max > right.max)
 result.max = left.max;
 else
 result.max = right.max;
 return result;
}
int main()
{
 int a[50], n, i;
 struct Pair result;
 printf("\n\tMinimum and Maximum using Divide and Conquer\n\n");
 printf("Enter the number of elements: ");
```

```
 scanf("%d", &n);
 printf("\nEnter the elements:\n");
 for (i = 0; i < n; i++)
 {
 printf("Element %d: ", i + 1);
 scanf("%d", &a[i]);
 }
 result = minMax(a, 0, n - 1);
 printf("\nMinimum element = %d", result.min);
 printf("\nMaximum element = %d\n", result.max);
 return 0;
}
```

output

```
PS C:\Users\admin\Desktop\AOA> gcc minmax.c -o minmax
PS C:\Users\admin\Desktop\AOA> .\minmax.exe
 Minimum and Maximum using Divide and Conquer
Enter the number of elements: 10
Enter the elements:
Element 1: 45
Element 2: 83
Element 3: 75
Element 4: 17
Element 5: 43
Element 6: 37
Element 7: 80
Element 8: 53
Element 9: 61
Element 10: 22
Minimum element = 17
Maximum element = 83
PS C:\Users\admin\Desktop\AOA>
```

Algo:

```
1   Algorithm MaxMin(i, j, max, min)
2   // a[1 : n] is a global array. Parameters i and j are integers,
3   // 1 ≤ i ≤ j ≤ n. The effect is to set max and min to the
4   // largest and smallest values in a[i : j], respectively.
5   {
6       if (i = j) then max := min := a[i]; // Small(P)
7       else if (i = j - 1) then // Another case of Small(P)
8       {
9           if (a[i] < a[j]) then
10          {
11              max := a[j]; min := a[i];
12          }
```

```
13        else
14        {
15            max := a[i]; min := a[j];
16        }
17    }
18    else
19    {   // If P is not small, divide P into subproblems.
20        // Find where to split the set.
21        mid := ⌊(i + j)/2⌋;
22        // Solve the subproblems.
23        MaxMin(i, mid, max, min);
24        MaxMin(mid + 1, j, max1, min1);
25        // Combine the solutions.
26        if (max < max1) then max := max1;
27        if (min > min1) then min := min1;
28    }
29  }
```

Exp 6

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>

struct Edge
{
   int source, destination, weight;
};

struct Graph
{
   int V, E;
   struct Edge *edge;
};

struct Graph* createGraph(int V, int E)
{
   struct Graph* graph = (struct Graph*) malloc(sizeof(struct Graph));
   graph->V = V;
   graph->E = E;
   graph->edge = (struct Edge*) malloc(graph->E * sizeof(struct Edge));
   return graph;
}

void FinalSolution(int dist[], int n)
{
   printf("\nVertex\tDistance from Source Vertex\n");
```

```
    for (int i = 0; i < n; ++i)
    {
        if (dist[i] == INT_MAX)
            printf("%d\t\tINF\n", i);
        else
            printf("%d\t\t%d\n", i, dist[i]);
    }
}

void BellmanFord(struct Graph* graph, int source)
{
    int V = graph->V;
    int E = graph->E;

    // Dynamically allocate StoreDistance to avoid VLA issues
    int *StoreDistance = (int*) malloc(V * sizeof(int));
    if (StoreDistance == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }

    /* Initialize all distances as infinity */
    for (int i = 0; i < V; i++)
        StoreDistance[i] = INT_MAX;

    /* Distance of source vertex from itself is 0 */
    StoreDistance[source] = 0;

    /* Relax all edges V - 1 times */
    for (int i = 1; i <= V - 1; i++)
    {
        for (int j = 0; j < E; j++)
        {
            int u = graph->edge[j].source;
            int v = graph->edge[j].destination;
            int weight = graph->edge[j].weight;
            if (StoreDistance[u] != INT_MAX &&
                StoreDistance[u] + weight < StoreDistance[v])
            {
                StoreDistance[v] = StoreDistance[u] + weight;
            }
        }
    }

    /* Check for negative weight cycle */
    for (int i = 0; i < E; i++)
    {
        int u = graph->edge[i].source;
```

```
        int v = graph->edge[i].destination;
        int weight = graph->edge[i].weight;
        if (StoreDistance[u] != INT_MAX && StoreDistance[u] + weight < StoreDistance[v])
        {
            printf("\nThis graph contains negative edge cycle\n");
            free(StoreDistance);
            return;
        }
    }

    FinalSolution(StoreDistance, V);
    free(StoreDistance);
    return;
}

int main()
{
    int V, E, S;
    printf("Enter number of vertices in graph:\n");
    scanf("%d", &V);
    printf("Enter number of edges in graph:\n");
    scanf("%d", &E);
    printf("Enter your source vertex number:\n");
    scanf("%d", &S);

    struct Graph* graph = createGraph(V, E);

    for (int i = 0; i < E; i++)
    {
        printf("\nEnter edge %d properties (source, destination, weight) respectively:\n", i
+ 1);
        scanf("%d", &graph->edge[i].source);
        scanf("%d", &graph->edge[i].destination);
        scanf("%d", &graph->edge[i].weight);
    }

    BellmanFord(graph, S);

    // Free allocated graph memory
    free(graph->edge);
    free(graph);

    return 0;
}
```

Output

```
PS C:\Users\admin\Desktop\AOA> gcc bellmanford.c -o bellmanford
```

```
PS C:\Users\admin\Desktop\AOA> .\bellmanford.exe
Enter number of vertices in graph
7
Enter number of edges in graph
10
Enter your source vertex number
0
Enter edge 1 properties source, destination, weight respectively
0
1
6
Enter edge 2 properties source, destination, weight respectively
0
2
5
Enter edge 3 properties source, destination, weight respectively
0
3
5
Enter edge 4 properties source, destination, weight respectively
1
4
-1
Enter edge 5 properties source, destination, weight respectively
2
1
-2
Enter edge 6 properties source, destination, weight respectively
2 4
1
Enter edge 7 properties source, destination, weight respectively
3
2
-2
Enter edge 8 properties source, destination, weight respectively
3
5
-1
Enter edge 9 properties source, destination, weight respectively
4
6
3
Enter edge 10 properties source, destination, weight respectively
5
6
3
Vertex Distance from Source Vertex
0 0
```

```
1 1
2 3
3 5
4 0
5 4
6 3
PS C:\Users\admin\Desktop\AOA>
```

Algo:

```
function bellmanFord(G, S)
for each vertex V in G
distance[V] <- infinite
previous[V] <- NULL
distance[S] <- 0
for each vertex V in G
for each edge (U,V) in G
tempDistance <- distance[U] + edge_weight(U, V)
if tempDistance < distance[V]
distance[V] <- tempDistance
for each edge (U,V) in G
If distance[U] + edge_weight(U, V) < distance[V}
Error: Negative Cycle Exists
return distance[]
```

Exp 7

```
#include <stdio.h>
int i, j, k, n, dist[10][10];
void floydWarshall()
{
 for (k = 0; k < n; k++)
 {
 for (i = 0; i < n; i++)
 {
 for (j = 0; j < n; j++)
 {
 if (dist[i][k] + dist[k][j] < dist[i][j])
 dist[i][j] = dist[i][k] + dist[k][j];
 }
 }
 }
}
int main()
{
 int i, j;
 printf("enter no of vertices :");
```

```
 scanf("%d", &n);
 printf("\n");
 for (i = 0; i < n; i++)
 {
 for (j = 0; j < n; j++)
 {
 printf("dist[%d][%d]:", i, j);
 scanf("%d", &dist[i][j]);
 }
 }
 floydWarshall();
 printf("\n\nshortest distances between every pair of vertices\n");
 for (i = 0; i < n; i++)
 {
 for (j = 0; j < n; j++)
 printf("%d\t", dist[i][j]);
 printf("\n");
 }
 return 0;
}
```

Output

```
PS C:\Users\Shifa\Desktop\AOA> gcc floyd.c -o floyd
PS C:\Users\Shifa\Desktop\AOA> .\floyd.exe
enter no of vertices :4
dist[0][0]:0
dist[0][1]:5
dist[0][2]:999
dist[0][3]:10
dist[1][0]:999
dist[1][1]:0
dist[1][2]:3
dist[1][3]:999
dist[2][0]:999
dist[2][1]:999
dist[2][2]:0
dist[2][3]:1
dist[3][0]:999
dist[3][1]:999
dist[3][2]:999
dist[3][3]:0
shortest distances between every pair of vertices
0 5 8 9
999 0 3 4
999 999 0 1
999 999 999 0
PS C:\Users\Shifa\Desktop\AOA>
```

Algo:

1. Start.
2. Read n and the distance matrix d.
3. Set INF for absent edges and 0 on diagonal. (In this code, enter 0 on diagonal.)
4. For each intermediate vertex k (0 to n - 1):
for each source vertex i:
for each destination vertex j:
• If d[i][k] + d[k][j] < d[i][j], update d[i][j].
5. Print the final distance matrix.
6. Stop

Exp 8

```
#include <stdio.h>
#include <stdlib.h> // Corrected: Changed from <math.h> to <stdlib.h> for the abs()
function

int board[20], count = 0;

void print(int n);
int place(int row, int column);
void queen(int row, int n);

int main()
{
   int n;
   printf("\n\t- N Queens Problem Using Backtracking -");
   printf("\n\nEnter number of Queens: ");
   if (scanf("%d", &n) != 1 || n <= 0 || n > 20) {
      printf("Invalid input. Please enter a number between 1 and 20.\n");
      return 1;
   }
   queen(1, n);
   return 0;
}

/* Function for printing the solution */
void print(int n)
{
   int i, j;
   printf("\n\nSolution %d:\n\n", ++count);
   for (i = 1; i <= n; i++)
      printf("\t%d", i);
   for (i = 1; i <= n; i++)
   {
      printf("\n\n%d", i);
```

```
        for (j = 1; j <= n; j++)
        {
            if (board[i] == j)
                printf("\tQ");
            else
                printf("\t-");
        }
    }
    printf("\n");
}

/* Function to check conflicts */
int place(int row, int column)
{
    int i;
    for (i = 1; i <= row - 1; i++)
    {
        if (board[i] == column)
            return 0;
        if (abs(board[i] - column) == abs(i - row))
            return 0;
    }
    return 1;
}

/* Function for proper positioning of queen */
void queen(int row, int n)
{
    int column;
    for (column = 1; column <= n; ++column)
    {
        if (place(row, column))
        {
            board[row] = column;
            if (row == n)
                print(n);
            else
                queen(row + 1, n);
        }
    }
}
```

Output

```
PS C:\Users\Shifa\Desktop\AOA> gcc nqueen.c -o nqueen
PS C:\Users\Shifa\Desktop\AOA> .\nqueen.exe
 - N Queens Problem Using Backtracking -
Enter number of Queens: 4
```

Solution 1:

```
 1 2 3 4
1 - Q - -
2 - - - Q
3 Q - - -
4 - - Q -
```

Solution 2:

```
 1 2 3 4
1 - - Q -
2 Q - - -
3 - - - Q
4 - Q - -
```

PS C:\Users\admin\Desktop\AOA>

Algo:

```
N - Queens (k, n)
{
For i ← 1 to n
do if Place (k, i) then
{
x [k] ← i;
if (k ==n) then
write (x [1....n]));
else
N - Queens (k + 1, n);
}
}
```

Exp 9

```
#include <stdio.h>
#include <stdlib.h>
int n = 4;
/* Calculate number of misplaced tiles */
int cal(int temp[10][10], int t[10][10])
{
 int i, j, m = 0;
 for (i = 0; i < n; i++)
 {
 for (j = 0; j < n; j++)
 {
 if (temp[i][j] != t[i][j])
 m++;
 }
 }
 return m;
```

```
}
/* Check whether current matrix is equal to target */
int check(int a[10][10], int t[10][10])
{
 int i, j, f = 1;
 for (i = 0; i < n; i++)
 {
 for (j = 0; j < n; j++)
 {
 if (a[i][j] != t[i][j])
 f = 0;
 }
 }
 return f;
}
void main() {
 int i, j, p;
 int a[10][10], t[10][10], temp[10][10], r[10][10];
 int x = 0, y = 0;
 int d = 1000, dmin = 0;
 int m = 0, l = 0;
 printf("\nEnter the matrix to be solved, space with zero :\n");
 for (i = 0; i < n; i++)
 {
 for (j = 0; j < n; j++)
 {
 scanf("%d", &a[i][j]);
 }
 }
 printf("\nEnter the target matrix, space with zero :\n");
 for (i = 0; i < n; i++)
 {
 for (j = 0; j < n; j++)
 {
 scanf("%d", &t[i][j]);
 }
 }
 printf("\nEntered Matrix is :\n");
 for (i = 0; i < n; i++)
 {
 for (j = 0; j < n; j++)
 printf("%d\t", a[i][j]);
 printf("\n");
 }
 printf("\nTarget Matrix is :\n");
 for (i = 0; i < n; i++)
 { for (j = 0; j < n; j++)
 printf("%d\t", t[i][j]);
```

```
printf("\n");
}
while (!check(a, t))
{
l++;
d = 1000;
/* Find position of blank (0) */
for (i = 0; i < n; i++)
{
for (j = 0; j < n; j++)
{
if (a[i][j] == 0)
{
x = i;
y = j;
}
}
}
/* Move Up */
if (x != 0)
{
for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
temp[i][j] = a[i][j];
p = temp[x][y];
temp[x][y] = temp[x - 1][y];
temp[x - 1][y] = p;
m = cal(temp, t);
dmin = l + m;
if (dmin < d)
{
d = dmin;
for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
r[i][j] = temp[i][j];
}
}
/* Move Down */
if (x != n - 1) {
for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
temp[i][j] = a[i][j];
p = temp[x][y];
temp[x][y] = temp[x + 1][y];
temp[x + 1][y] = p;
m = cal(temp, t);
dmin = l + m;
if (dmin < d)
```

```
{
d = dmin;
for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
r[i][j] = temp[i][j];
}
}
/* Move Right */
if (y != n - 1)
{
for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
temp[i][j] = a[i][j];
p = temp[x][y];
temp[x][y] = temp[x][y + 1];
temp[x][y + 1] = p;
m = cal(temp, t);
dmin = l + m;
if (dmin < d)
{
d = dmin;
for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
r[i][j] = temp[i][j];
}
}
/* Move Left */
if (y != 0)
{
for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
temp[i][j] = a[i][j]; p = temp[x][y];
temp[x][y] = temp[x][y - 1];
temp[x][y - 1] = p;
m = cal(temp, t);
dmin = l + m;
if (dmin < d)
{
d = dmin;
for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
r[i][j] = temp[i][j];
}
}
/* Copy best matrix to current matrix */
for (i = 0; i < n; i++)
{
for (j = 0; j < n; j++)
```

```
 {
 a[i][j] = r[i][j];
 temp[i][j] = 0;
 }
 }
 printf("\nCalculated Intermediate Matrix Value :\n");
 for (i = 0; i < n; i++)
 {
 for (j = 0; j < n; j++)
 printf("%d\t", r[i][j]);
 printf("\n");
 }
 }
 printf("\nMinimum cost : %d\n", l);
}
```

output

```
PS C:\Users\Shifa\Desktop\AOA> gcc puzzle.c -o puzzle
PS C:\Users\Shifa\Desktop\AOA> .\puzzle.exe
Enter the matrix to be solved, space with zero :
1 2 3 4
5 6 0 8
9 10 7 11
13 14 15 12
Enter the target matrix, space with zero :
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
Entered Matrix is :
1 2 3 4
5 6 0 8
9 10 7 11
13 14 15 12
Target Matrix is :
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
Calculated Intermediate Matrix Value :
1 2 3 4
5 6 7 8
9 10 0 11
13 14 15 12
Calculated Intermediate Matrix Value :
1 2 3 4
5 6 7 8
```

```
9 10 11 0
13 14 15 12
Calculated Intermediate Matrix Value :
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
Minimum cost : 3
PS C:\Users\Shifa\Desktop\AOA>
```

Exp 10

```
#include <stdio.h>
#include <string.h>
int flag;
void naive_search(char *pattern, char *text)
{
 int m = strlen(pattern);
 int n = strlen(text);
 int i, j;
 flag = 1;
 for (i = 0; i <= n - m; i++)
 {
 for (j = 0; j < m; j++)
 {
 if (text[i + j] != pattern[j])
 break;
 }
 if (j == m)
 {
 flag = 0;
 printf("Pattern found at index %d\n", i);
 }
 }
}
void rabin_karp_search(char *pattern, char *text, int q)
{
 int m = strlen(pattern);
 int n = strlen(text);
 int i, j;
 int p = 0;
 int t = 0;
 int h = 1; flag = 1;
 /* Calculate h = d^(m-1) % q */
 for (i = 0; i < m - 1; i++)
 h = (h * 256) % q;
 /* Calculate initial hash values */
 for (i = 0; i < m; i++)
```

```
{
p = (256 * p + pattern[i]) % q;
t = (256 * t + text[i]) % q;
}
/* Slide pattern over text */
for (i = 0; i <= n - m; i++)
{
if (p == t)
{
for (j = 0; j < m; j++)
{
if (text[i + j] != pattern[j])
break;
}
if (j == m)
{
flag = 0;
printf("Pattern found at index %d\n", i);
}
}
/* Calculate hash for next substring */
if (i < n - m)
{
t = (256 * (t - text[i] * h)
+ text[i + m]) % q;
if (t < 0)
t = t + q;
}
}
} int main()
{
 int choice, q;
 char pattern[100], text[100];
 while (1)
 {
 printf("\nString Matching Algorithms:\n");
 printf("1. Naive String Matching\n");
 printf("2. Rabin-Karp Algorithm\n");
 printf("3. Exit\n");
 printf("Enter your choice: ");
 scanf("%d", &choice);
 switch (choice)
 {
 case 1:
 printf("Enter the text: ");
 scanf("%s", text);
 printf("Enter the pattern: ");
 scanf("%s", pattern);
```

```
 naive_search(pattern, text);
 if (flag == 1)
 printf("No match found\n");
 break;
 case 2:
 printf("Enter the text: ");
 scanf("%s", text);
 printf("Enter the pattern: ");
 scanf("%s", pattern);
 printf("Enter a prime number: "); scanf("%d", &q);
 rabin_karp_search(pattern, text, q);
 if (flag == 1)
 printf("No match found\n");
 break;
 case 3:
 printf("Exiting program.\n");
 return 0;
 default:
 printf("Invalid choice. Try again.\n");
 }
 }
 return 0;
}
```

output

```
Naive String Matching
PS C:\Users\Shifa\Desktop\AOA> gcc string.c -o string
PS C:\Users\Shifa\Desktop\AOA> .\string.exe
String Matching Algorithms:
1. Naive String Matching
2. Rabin-Karp Algorithm
3. Exit
Enter your choice: 1
Enter the text: AABAACAADAABAABA
Enter the pattern: AABA
Pattern found at index 0
Pattern found at index 9
Pattern found at index 12
String Matching Algorithms:
1. Naive String Matching
2. Rabin-Karp Algorithm
3. Exit
Enter your choice: 3
Exiting program.
PS C:\Users\Shifa\Desktop\AOA>
Rabin-Karp
PS C:\Users\Shifa\Desktop\AOA> .\string.exe
```

```
String Matching Algorithms:
1. Naive String Matching
2. Rabin-Karp Algorithm
3. Exit
Enter your choice: 2
Enter the text: AABAACAADAABAABA
Enter the pattern: AABA
Enter a prime number: 101 Pattern found at index 0
Pattern found at index 9
Pattern found at index 12
String Matching Algorithms:
1. Naive String Matching
2. Rabin-Karp Algorithm
3. Exit
Enter your choice: 3
Exiting program.
PS C:\Users\Shifa\Desktop\AOA>
```

Algo:

```
Naive (T[1...n], P[1...m])
{
   For(s=1 to n-m) do
   {
      If(P[1...m] = T[s...s+m]) then
         Print("Pattern finding with shift", s);
   }
}
compute hp (for pattern p)
compute ht (for the first substring of t with m length)

for i = 1 to n-m
{
   if hp = ht
   {
      match t with P,
      if matched
         return 1
      else
         t = (d * (t - txt[i] * h) + txt[i + M]) % q
   }
}
```